

LumiDrive / RoadSight Research Project Handbook

Transition-Aware Cross-Task Consistency for Robust Multi-Task Road Perception under
Changing Weather Conditions

Research and implementation reference | September 2026

LumiDrive / RoadSight Research Project Handbook

```
@page { size: A4; margin: 18mm; }
body { font-family: Georgia, serif; color:#202735; line-height:1.45; max-width:900px; margin:auto; }
h1 { color:#123b52; font-size:28px; margin-bottom:4px; }
h2 { color:#176b72; border-bottom:1px solid #b7d4d0; padding-bottom:4px; margin-top:28px; }
h3 { color:#24546a; margin-top:20px; }
.subtitle { color:#55717a; font-size:16px; }
.notice { background:#fff4d8; border-left:5px solid #d69d2f; padding:12px 16px; }
.box { background:#eef7f6; padding:12px 16px; border:1px solid #c7dfdb; }
table { width:100%; border-collapse:collapse; margin:12px 0; font-size:13px; }
th { background:#176b72; color:white; text-align:left; }
th, td { border:1px solid #cbd8da; padding:7px; vertical-align:top; }
code { font-family: Consolas, monospace; font-size: .9em; }
li { margin:4px 0; }
.equation { text-align:center; font-family: Georgia, serif; font-size:17px; margin:14px; }
.small { color:#5d6c70; font-size:12px; }
```

LumiDrive / RoadSight Research Project Handbook

Transition-Aware Cross-Task Consistency for Robust Multi-Task Road Perception under Changing Weather Conditions

Research and implementation reference | September 2026

Evidence status: The repository contains a substantial PyTorch prototype, synthetic sequence data, a CULane adapter, training and evaluation scripts, video inference, and a React dashboard. It does not contain trained checkpoints, experiment logs, or verified numerical results. Current manuscript numbers must not be presented as measured results until reproduced.

1. Project Summary

The project studies whether lightweight temporal memory and an explicit cross-task consistency objective can stabilize lane and drivable-area predictions during changing visibility without losing practical inference speed.

The system accepts forward-facing monocular RGB video and is designed to predict lane segmentation, drivable-area segmentation, and an object-detection branch. The current training path supervises only lane and drivable-area tasks. The browser dashboard is a separate heuristic demonstration and is not connected to the PyTorch model.

2. Research Problem

Lane markings are thin, high-frequency structures that are vulnerable to faded paint, broken markings, shadows, rain, fog, glare, night illumination, water spray, and temporary occlusion. Drivable regions may remain plausible after lane evidence has degraded.

- Asynchronous task failure: lane predictions disappear while drivable predictions remain active, creating contradictory outputs.

- Temporal flicker: adjacent frames produce rapidly changing masks and unstable lane geometry.

Research question: Can short-term temporal context and lane/drivable consistency reduce these failures during weather transitions?

Scope: monocular RGB perception, lane and drivable segmentation, short temporal windows, video evaluation, consistency metrics, and deployment measurements. The scope excludes direct vehicle control, LiDAR/radar fusion, safety certification, and claims of universal weather robustness.

3. Research Gap and Literature Position

Existing research already covers panoptic multi-task perception, temporal road perception, cross-task relation learning, uncertainty weighting, adverse-weather robustness, and transitional-weather segmentation. The defensible gap is narrower:Ⓓ

Weather-transition-induced cross-task inconsistency in driving perception, measured with task accuracy, temporal instability, spatial contradiction, worst-condition degradation, and efficiency.Ⓓ

Related directionWhat it establishesWhat must not be claimed as newYOLOP / related MTLJoint object, drivable-area, and lane perception.Basic three-task MTL.SHIFTContinuous weather and domain shifts.Continuous-weather data itself.Cross-task relation learningExplicit relations between road-perception tasks.Task relation modeling alone.Advent / UW-MTL / RMT-PPADTemporal, uncertainty-aware, and robust MTL approaches.Generic temporal or weather-adaptive fusion.CaRS / TWFNet / TwinMixingTransition weather, forecasting, and lightweight adverse-weather MTL.Transitional-weather perception or lightweight MTL alone.

Safe wording: “We propose and evaluate a lightweight temporal multi-task framework for analyzing and reducing cross-task inconsistency during changing visibility.” Avoid “first-ever” claims until a final literature and patent search is complete.

4. Research Questions and Hypotheses

- Does temporal memory improve lane IoU and F1 in difficult conditions?
- Does it reduce frame-to-frame lane-mask flicker?
- Does the consistency regularizer reduce lane pixels outside the predicted drivable region?
- Are improvements retained on clear and steady adverse-weather data?
- What are the accuracy, latency, FPS, parameter, and memory costs?
- Can calibrated confidence identify unreliable predictions?

Primary hypothesis: temporal context plus cross-task regularization improves transition robustness and stability over an otherwise identical single-frame baseline.

5. Implemented Architecture

Implementation: python_pipeline/models/.

- Input sequence: [B, T, 3, H, W], with default T=4.
- Custom lightweight CSP-style convolutional backbone.
- Three-scale FPN with P3, P4, and P5, each using 128 output channels.
- ConvGRU temporal aggregation at P3.
- Spatial gate that fuses current features and hidden memory.
- Lane segmentation head: two classes.
- Drivable-area head: background, main drivable, alternate drivable.
- Object heads at P3, P4, and P5.

Backbone scales: C3 has 64 channels at 1/8 resolution, C4 has 128 channels at 1/16, and C5 has 256 channels at 1/32.

ConvGRU recurrence:
$$z_t, r_t = \text{sigmoid}(\text{Conv}([x_t, h_{t-1}]))$$
$$h_t = (1 - z_t) h_{t-1} + z_t \tanh(\text{Conv}([x_t, r_t h_{t-1}]))$$

The object head is architectural only: object labels, loss, decoder, NMS, and evaluation are missing.

6. Training Objective

Lane loss: $L_{\text{lane}} = L_{\text{focal}} + L_{\text{Dice}}$

Drivable loss: $L_{\text{driv}} = L_{\text{cross-entropy}} + L_{\text{Dice}}$

Cross-task consistency penalizes lane probability outside a dilated drivable region: $L_{\text{CTC}} = \text{mean}[M_{\text{lane}}(p) (1 - \text{Dilate}(M_{\text{driv}}(p), k))]$

Temporal flicker loss: $L_{\text{temp}} = \text{mean} |M_t(p) - M_{t-1}(p)|$

Implemented total: $L_{\text{total}} = 1.5 L_{\text{lane}} + 1.0 L_{\text{driv}} + 0.6 L_{\text{CTC}} + 0.4 L_{\text{temp}}$

The temporal comparison is unaligned and does not compensate for ego-motion. The paper must either add alignment or state this limitation clearly.

7. Data

Synthetic sequence data

`python_pipeline/data/dataset.py` generates 640 x 360 road scenes with clear, rain, fog, night glare, and a clear-to-rain transition. It generates lane and three-class drivable masks. Defaults are 160 training samples, 40 validation samples, 50 evaluation samples, and four-frame sequences. Synthetic data validates the pipeline but cannot prove real-world weather robustness.

CULane adapter

`python_pipeline/data/culane_dataset.py` loads contiguous frames and genuine lane masks. CULane has no drivable-area ground truth in this implementation; its drivable target is a geometric proxy and must not be reported as genuine drivable accuracy.

Recommended real-data plan

Use CULane or VIL-100 for temporal lane evaluation. Use BDD100K or another dataset with valid lane and drivable labels for genuine MTL. Investigate SHIFT, CaRS, or another validated transition dataset for weather-transition evaluation. Split by complete video, route, or scene before creating temporal windows.

8. Training and Evaluation

Training is in `python_pipeline/train.py`. It uses AdamW, learning rate $1e-3$, weight decay $1e-4$, batch size 4, cosine annealing, optional CUDA mixed precision, validation every five epochs, and best-checkpoint selection from lane IoU and drivable mIoU.

Evaluation is in `python_pipeline/evaluate.py`. Current metrics are lane IoU, lane F1, drivable mIoU, CTIR, and raw pixelwise flicker across overall, clear, rain, fog, night-glare, and transition groups.

CTIR: $\text{CTIR} = |\text{predicted lane pixels outside predicted drivable area}| / |\text{predicted lane pixels}| \times 100$

Flicker: $\text{Flicker} = |M_t \text{ XOR } M_{t-1}| / |P| \times 100$

Missing evaluation components include precision, recall, ECE, reliability diagrams, optical-flow alignment, FPS benchmarking, latency methodology, FLOPs, parameter count, memory, confidence intervals, and repeated seeds.

9. Required Baselines and Ablations

Variant Purpose **Single-frame baseline** Reproducible starting point. **Baseline + ConvGRU** Temporal aggregation effect. **Baseline + CTC** Cross-task regularization effect. **Baseline + temporal loss** Flicker-loss effect. **Full model** Combined performance. **Sequence lengths 1, 3, 4, 5** Accuracy versus compute trade-off. **Real versus synthetic augmentation** Transfer to real conditions.

All variants must use the same test set, preprocessing, evaluation code, and hardware protocol.

10. Video Inference and Frontend

`python_pipeline/inference_video.py` reads MP4, AVI, and MOV files, maintains a sliding buffer, runs the model, writes lane and drivable overlays, estimates approximate lateral offset, and exports CSV telemetry. The offset assumes a 3.7 m lane width and is not metric without camera calibration.

The Vite/React application in `src/` provides architecture exploration, benchmark views, telemetry graphs, paper viewing, synthetic scenarios, upload, and webcam controls. Its browser engine uses heuristic image analysis and synthetic telemetry. It is a demonstration layer, not a PyTorch benchmark.

11. Evidence Audit

ComponentStatusBackbone, FPN, ConvGRUImplemented.Lane and drivable headsImplemented.Object headImplemented architecturally; not trained.Consistency and flicker lossesImplemented; flicker is unaligned.Synthetic weather dataImplemented.CULane adapterImplemented; external data required.Training and evaluationImplemented, limited.Video inferenceImplemented.Geometry fitting and confidence calibrationMissing.Real transition dataset, checkpoints, logsMissing.Verified numerical resultsMissing.

12. Publication Warnings

The current manuscript reports 64.2% inconsistency reduction, +13.8% worst-case F1, and 39.8 FPS on an edge GPU, but the repository contains no artifacts proving these numbers. The manuscript table reports transition F1 increasing from 64.8 to 81.7, an absolute change of 16.9 points, and CTIR decreasing from 14.8% to 4.2%, approximately a 71.6% relative reduction. These figures conflict and their origin is undocumented.

Until experiments are run, all such values must be removed or labelled as targets. The paper must not claim trained object perception, calibrated reliability, real-weather robustness, or safety certification without evidence.

13. Limitations

- Synthetic weather is not equivalent to real weather.
- CULane is not an adverse-weather or Indian-road benchmark.
- CULane drivable labels are proxies.
- Monocular RGB has depth and geometry ambiguity.
- Temporal metrics are not motion compensated.
- Object detection is not trained.
- Confidence is not calibrated.
- No real vehicle control or safety validation exists.
- No camera calibration is implemented.
- The frontend is heuristic and separate from PyTorch inference.
- No trained checkpoint or experiment log is included.

14. Publication Checklist

- Freeze the final scope and decide whether objects are included.

- Select valid datasets and document licenses.
- Create video-level or route-level splits.
- Reproduce a single-frame baseline.
- Train temporal and consistency variants with fixed seeds.
- Run all ablations and sequence-length tests.
- Add motion compensation or document its absence.
- Measure accuracy, CTIR, flicker, calibration, FPS, latency, FLOPs, parameters, and memory.
- Save checkpoints, logs, configurations, environment versions, and result tables.
- Include qualitative success and failure cases.
- Remove unsupported claims.
- Perform the final literature and patent search.

15. Repository Map

- paper/main.tex: current IEEE manuscript draft.
- paper/references.bib: bibliography.
- python_pipeline/models/: backbone, FPN, model, and ConvGRU.
- python_pipeline/losses/consistency_loss.py: losses.
- python_pipeline/data/: synthetic and CULane data.
- python_pipeline/train.py: training.
- python_pipeline/evaluate.py: evaluation.
- python_pipeline/inference_video.py: video inference.
- src/: React dashboard.
- docs/extracted/: research gap, temporal lane plan, and implementation guide.

Conclusion

LumiDrive / RoadSight has a coherent prototype foundation. Its strongest publishable story is a controlled study of temporal stabilization and lane/drivable consistency under changing visibility. The decisive next step is experimental validation using named datasets, fixed splits, saved checkpoints, logged configurations, and repeatable evaluation.